

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

Name of the Student	U. Somesh Kumar
Reg. No	192425019

COURSE NAME: Deep Learning
FACULTY: Dr. Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

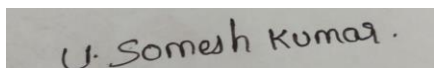
Duration: 30 minutes

Marks: 25

Code of Conduct:

I U. Somesh Kumar (192425019) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

U. Somesh Kumar
Name of the student:

QUESTIONS:05

Total Marks:25

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:

Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Answer: **1. Learning Algorithms (Optimization Behaviour)**

Given:

Gradient Descent update rule:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

where:

- θ = Model Parameters
- η = Learning Rate
- $J(\theta)$ = Loss Function

Analysis:

Consider a convex loss function:

$$J(\theta) = \theta^2$$

Gradient:

$$\nabla J(\theta) = 2\theta$$

Update Rule:

$$\begin{aligned}\theta_{t+1} &= \theta_t - 2\eta\theta_t \\ \theta_{t+1} &= (1 - 2\eta)\theta_t\end{aligned}$$

For a high learning rate:

$$\eta = 0.6$$

$$1 - 2(0.6) = -0.2$$

The model moves rapidly toward the optimum.

Learning Rate Decay:

$$\eta_t = \frac{\eta_0}{1 + kt}$$

As t increases:

$$\eta_t \downarrow$$

Small updates help fine-tune the solution near the minimum.

Risk:

If

$$\eta > 1$$

the algorithm may overshoot the minimum and diverge.

Conclusion:

A high initial learning rate provides faster convergence, while learning rate decay ensures stable optimization and better final accuracy.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:

Derive the Maximum Likelihood Estimator for μ , and analytically explain how the estimate changes when:

- The dataset contains outliers
 - The sample size increases
- What does this imply about the robustness of MLE?

Answer: **Given:**

$$x_1, x_2, \dots, x_n \sim N(\mu, \sigma^2)$$

Likelihood Function:

$$L(\mu) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x_i - \mu)^2}{2\sigma^2}}$$

Taking Log-Likelihood:

$$\log L(\mu) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

Differentiate with respect to μ :

$$\frac{d}{d\mu} \log L(\mu) = \frac{1}{\sigma^2} \sum_{i=1}^n (x_i - \mu)$$

Set equal to zero:

$$\begin{aligned} \sum_{i=1}^n (x_i - \mu) &= 0 \\ n\mu &= \sum_{i=1}^n x_i \end{aligned}$$

Therefore,

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$$

Effect of Outliers:

Dataset:

$$\begin{aligned} &\{10, 11, 12, 13, 14\} \\ &\hat{\mu} = 12 \end{aligned}$$

After adding outlier 100:

$$\begin{aligned} &\{10, 11, 12, 13, 14, 100\} \\ &\hat{\mu} = \frac{160}{6} = 26.67 \end{aligned}$$

The estimate changes significantly.

Effect of Increasing Sample Size:

Variance of estimator:

$$\text{Var}(\hat{\mu}) = \frac{\sigma^2}{n}$$

As

$$n \uparrow \\ \text{Var}(\hat{\mu}) \downarrow$$

Conclusion:

MLE becomes more accurate with larger sample sizes but is sensitive to outliers.

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:

Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Answer: **Given:**

Number of Features:

$$d = 10000$$

Number of Samples:

$$n = 500$$

Since:

$$d \gg n$$

the dataset suffers from the Curse of Dimensionality.

Proposed Pipeline:

Step 1: Feature Selection

Reduce features:

$$10000 \rightarrow 200$$

Step 2: PCA (Dimensionality Reduction)

Reduce dimensions further:

$$200 \rightarrow 50$$

while retaining approximately 95% variance.

Step 3: Regularization

Apply L2 Regularization:

$$J(w) = Loss + \lambda ||w||^2$$

Step 4: Classification

Train Logistic Regression or SVM.

Result:

Reduced dimensions decrease computational complexity and overfitting.

Conclusion:

Feature selection, PCA, and regularization improve model generalization and performance.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:

Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Answer: **Single Hidden Neuron:**

$$y = \sigma(w_1x + b_1)$$

Multiple Hidden Neurons:

$$y = \sum_{i=1}^m v_i \sigma(w_i x + b_i)$$

where:

m = Number of Hidden Neurons

Analysis:

For:

$$m = 10$$

the network combines 10 nonlinear functions.

For:

$$m = 100$$

the network combines 100 nonlinear functions.

Representational power increases significantly.

Parameter Comparison:

Assume:

Input Neurons = 20

Output Neurons = 1

For 10 Hidden Neurons:

$$(20 \times 10) + (10 \times 1) = 210$$

For 100 Hidden Neurons:

$$(20 \times 100) + (100 \times 1) = 2100$$

Increase Factor:

$$\frac{2100}{210} = 10$$

Effect:

- Higher learning capacity
- Higher computational cost
- Greater risk of overfitting

Conclusion:

Increasing hidden neurons improves representation capability but may reduce generalization if the network becomes too complex.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:

Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

Answer: **SGD Update Rule:**

$$w_{t+1} = w_t - \eta \nabla J(w_t)$$

Impact on Gradient Updates:

Gradient Vector:

$$\nabla J(w) = \left(\frac{\partial J}{\partial w_1}, \frac{\partial J}{\partial w_2}, \dots, \frac{\partial J}{\partial w_d} \right)$$

As dimensionality increases:

$$d \uparrow$$

Gradient calculations become more complex and noisier.

Impact on Convergence Speed:

Distance Formula:

$$Distance = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}$$

As dimensions increase:

$$Distance \uparrow$$

SGD requires more iterations to converge.

Impact on Generalization:

Required samples:

$$n \propto d$$

When:

$$d \gg n$$

the model overfits the training data.

Solution 1: PCA

Reduce dimensions:

$$10000 \rightarrow 100$$

Solution 2: Regularization

Apply:

$$J(w) = Loss + \lambda ||w||^2$$

to penalize large weights.

Conclusion:

Dimensionality reduction and regularization improve gradient stability, accelerate convergence, and enhance model generalization.